

Kapitel 1: Kom igång med Python

Viktiga Begrepp och Studieförfrågor (DA2005)

Studieguide

July 21, 2026

1 Kapitel 1: Intro till Python

Programmering

Att skriva instruktioner (program) som en dator sedan kan köra för att lösa specifika problem.

Anaconda

En distribution och ett verktygsprogram för Python som används för att hantera paket, bibliotek och virtuella utvecklingen.

Spyder

En integrerad utvecklingsmiljö (IDE) som följer med Anaconda, utrustad med kodredigering, syntaxmarkering (syntax highlighting) och en interaktiv konsol.

Python-tolk (Interpreter)

Det program som läser, analyserar och kör (tolkar) Python-kod rad för rad i realtid.

Konsol (Console)

Det interaktiva fönstret (t.ex. i Spyder) där man kan skriva enstaka Python-uttryck för att direkt se deras resultat, samt där programmets utskrifter visas.

Variabel

En namngiven behållare i datorns minne som används för att spara och referera till ett värde.

Tilldelning (Assignment)

Processen där ett värde lagras i en variabel med hjälp av tilldelningsoperatoren `=`. Exempel:
`s = 67`.

Datatyp

En klassificering av data som talar om för tolken hur värdet ska användas. De tre primära typerna i kapitel 1 är:

- **Heltal (int):** Representerar heltal utan decimaler (t.ex. `17`, `-3`).
- **Flyttal (float):** Representerar reella tal med decimalpunkt (t.ex. `3.14`, `1.25`).
- **Sträng (str):** En sekvens av tecken som omges av enkelfnuttar (`'hej'`) eller dubbelfnuttar (`"hej"`).

Aritmetiska operationer

Matematiska operationer för beräkningar:

- **Heltalsdivision (//):** Dividerar två tal och avrundar nedåt till närmaste heltal.
- **Exponentiering (**):** Upphöjt till (t.ex. `2**3` ger `8`).

- **Modulus/Rest (%)**: Returnerar resten vid en heltalsdivision (t.ex. `5 % 2` ger 1).

Reserverade ord (Keywords)

Ord som har en specifik fördefinierad betydelse i Python och som därför inte får användas som variabelnamn (t.ex. `and`, `if`, `for`, `True`).

Identifierare (Identifiers)

Namn på resurser i programmet, såsom variabler, funktioner eller klasser. De måste börja med en bokstav eller understreck (`_`) och får inte börja med en siffra.

Litteral

Ett fast, konstant värde som skrivs direkt i programkoden (t.ex. talet `2.5` eller strängen `'SU'`). Bör helst inte spridas okontrollerat i koden (undvik s.k. "hårdkodning").

Instuderingsfrågor (Review Questions)

Fråga 1: Vad är skillnaden mellan en Python-tolk och en utvecklingsmiljö som Spyder?

Svar: Tolken kör själva koden, medan Spyder är ett verktyg/redigeringsprogram som gör det lättare för programmeraren att skriva, felsöka och organisera koden.

Fråga 2: Varför är ordningsföljden på instruktioner viktig vid tilldelning av variabler? Ge ett exempel på en otillåten ordning.

Svar: Python körs sekventiellt (rad för rad). Du kan inte använda en variabel i en beräkning innan den har definierats. Till exempel ger följande kod felmeddelandet `NameError`:

```
v = s / t # Fel! s och t är inte definierade än.
s = 67
t = 1.25
```

Fråga 3: Förklara skillnaden i resultat mellan operatorerna `/`, `//`, och `%` med hjälp av talen 7 och 3.

Svar:

- `7 / 3` ger flyttalet 2.3333333333333335 (vanlig division).
- `7 // 3` ger heltalet 2 (heltalsdivision, kapar decimalerna).
- `7 % 3` ger resten 1 (modulus, eftersom $7 = 2 \times 3 + 1$).

Fråga 4: Vad händer om du kör koden `'hej' * 3` respektive `'hej' + 3`? Förklara varför.

Svar:

- `'hej' * 3` duplicerar strängen och ger `'hejhejhej'`. Detta är en tillåten operation (sträng multiplicerat med heltal).
- `'hej' + 3` ger ett `TypeError` eftersom man inte kan addera (konkatenera) en sträng med ett heltal (`int`) direkt utan att först konvertera talet till en sträng.

Fråga 5: Vilka av följande är giltiga identifierare (variabelnamn) i Python? Om de är ogiltiga, förklara varför.

a) `my_variable`

- b) `2nd_value`
- c) `class`
- d) `Value_17`

Svar:

- a) och d) är **giltiga**.
- b) är **ogiltig** eftersom en identifierare inte får börja med en siffra.
- c) är **ogiltig** eftersom `class` är ett reserverat ord (keyword) i Python.

Fråga 6: Vad menas med att “hårdkoda” ett värde, och varför bör man undvika litteraler spridda i koden enligt kompendiet?

Svar: Att hårdkoda innebär att skriva in konstanta värden (litteraler) direkt i beräkningar spridda över programmet. Det bör undvikas eftersom det försvårar uppdateringar (om värdet ändras måste man leta upp alla ställen manuellt) och ökar risken för programmeringsfel. Det är bättre att definiera värdet en gång i en tydligt namngiven variabel längst upp i koden.

2 Kapitel 2: intro till python

Syntax

Regler för hur kod skrivs för att Python ska förstå den.

Semantik

Innebörden eller betydelsen av vad koden gör.

Datatyper

Python har inbyggda typer som `str`, `int`, `float`, `complex`, `bool` och `NoneType`.

Typomvandling

Kan vara implicit (automatisk) eller explicit (t.ex. `int(x)`).

Kortslutning (Short circuiting)

När logiska operatorer (`and/or`) avbryter utvärderingen så fort resultatet är garanterat.

Modulär kod

Program uppdelade i mindre, fristående funktioner.

Docstrings

Dokumentationssträngar knutna till funktioner, läsbara via `help()`.

Scope

Räckvidden för en variabel (global eller lokal).

Fråga 1: Vad är skillnaden mellan `SyntaxError` och semantiska fel?

Svar: Syntaxfel innebär att koden inte följer språkets grammatik, vilket gör att den inte kan köras. Semantiska fel innebär att koden är korrekt skriven men utför fel logik, vilket resulterar i oväntade utdata.

Fråga 2: Varför representeras flyttal som approximationer?

Svar: Python representerar flyttal genom bråk i bas 2. Det är matematiskt omöjligt att representera alla reella tal exakt med ett begränsat antal bitar på en dator.

Fråga 3: Hur ändrar man en global variabel inuti en funktion?

Svar: Genom att använda det reserverade ordet **global** inuti funktionen deklarerar man att man avser att ändra den globala variabeln snarare än att skapa en lokal variabel med samma namn.

Fråga 4: Vad händer om en funktion saknar return?

Svar: Om **return** saknas returnerar funktionen värdet **None** som standard.

Fråga 5: Varför är `elif` användbart?

Svar: Det förbättrar kodens läsbarhet genom att ersätta krångliga, nästlade **if-else**-strukturer, vilket gör logiken tydligare att följa.

3 Kapitel 3: Listor och iteration

Listor

En sekvens av element (av valfri typ) inneslutna i hakparenteser `[]`.

Iteration

Processen att automatisera repetitiva uppgifter.

for-loop

Används för att iterera över element i en sekvens.

while-loop

Används för att repetera kod så länge ett villkor är sant.

Fråga 1: Varför är listor viktiga?

Svar: De gör det möjligt att samla och hantera flera element effektivt, vilket underlättar beräkningar och databehandling.

Fråga 2: Vad är skillnaden mellan `for` och `while`?

Svar: **for**-loopar används för att iterera över samlingar, medan **while**-loopar repeterar kod baserat på ett sant villkor.

4 Kapitel 4: Filer och dictionaries

Datastruktur

En struktur för att organisera och samla data logiskt.

Uppslagstabell

En datastruktur som kan indexeras med andra typer än bara heltal.

Filhantering

Tekniker för att läsa från och skriva till filer på disk.

Besvarade Instuderingsfrågor

Fråga 1: Vad skiljer en uppslagstabell från en lista?

Svar: Listor indexeras via heltal, medan uppslagstabeller tillåter indexering via andra datatyper.

Fråga 2: Vad är syftet med filhantering?

Svar: Det möjliggör permanent lagring av data samt inläsning av extern data som inte finns direkt i programkoden.

5 Kapitel 5: Felhantering och undantag

Felhantering

Metoder för att hantera programfel (t.ex. krascher).

Särfall (Exception)

Signalering av fel som uppstått under körning.

try/except

Pythons verktyg för att fånga och hantera uppkomna fel.

Listomfattning

Kompakt syntax för att skapa och bearbeta listor.

Fråga 1: Varför är särfall bättre än felkoder?

Svar: Felkoder gör koden svåräst och komplex p.g.a. ständiga kontroller. Särfall separerar felhantering från den ordinarie logiken.

Fråga 2: Vad är fördelen med listomfattning?

Svar: Det möjliggör mer kompakt och effektiv kod vid skapande av listor jämfört med traditionella loopar.

6 Kapitel 6: Sekvenser och generatorer

Sekvenstyp

Datatyp med gemensamma operationer (t.ex. indexering, slice).

Muterbar

Objekt som kan ändras efter skapande (t.ex. listor).

Icke-muterbar

Objekt som är konstanta (t.ex. tupler, strängar).

Fråga 1: Vad är skillnaden mellan muterbara och icke-muterbara objekt?

Svar: Muterbara objekt kan ändras efter skapande, medan icke-muterbara objekt är konstanta.

Fråga 2: Varför är muterbarhet viktigt?

Svar: Det påverkar hur objekt hanteras i funktioner och kan leda till oväntade sideffekter om man inte är medveten om att originalobjektet ändras.

7 Kapitel 7: Moduler, bibliotek och programstruktur

Sekvenstyp

Datatyp med gemensamma operationer (t.ex. indexering, slice).

Muterbar

Objekt som kan ändras efter skapande (t.ex. listor).

Icke-muterbar

Objekt som är konstanta (t.ex. tupler, strängar).

Fråga 1: Vad är skillnaden mellan muterbara och icke-muterbara objekt?

Svar: Muterbara objekt kan ändras efter skapande, medan icke-muterbara objekt är konstanta.

Fråga 2: Varför är muterbarhet viktigt?

Svar: Det påverkar hur objekt hanteras i funktioner och kan leda till oväntade sideffekter om man inte är medveten om att originalobjektet ändras.

```
'''
short/modular (variables, functions, files)
variables (nouns, repeating stuff should be global constants)
functions (action on what, return for local > global=constant, keep
    be math alone, don't return int/bool/fixed stuff/unspecified
    numbers, several returns > one)
documentation (what, arg & returns - name, type, description)
OOP (code skeleton)
Tests
! main
commenting (why you chose a method, use this when) - no syntax
    explanations, no examples, no long sentences

Type hinting (def(arg: type) -> return_type)
'''
```

8 Kapitel 8: Funktionell programmering

Funktionell programmering

bygger på att man anropar, sätter ihop och modifierar funktioner.

Rekursion

En teknik där en funktion anropar sig själv för att lösa ett problem.

Högre ordningens funktioner

Funktioner som tar andra funktioner som argument eller returnerar funktioner.

Anonyma funktioner (lambda)

Funktioner utan namn som definieras med `lambda`-syntaxen. Används ofta som argument till högre ordningens funktioner då de inte behöver återanvändas.

Svansrekursion

En form av rekursion där det rekursiva anropet är det sista som sker i funktionen. Vissa språk kan optimera detta till en loop för att spara minne.

Fråga 1: Vad innebär rekursion?

Svar: Rekursion är när en funktion anropar sig själv. Det är ett alternativ till loopar och kan göra koden mer elegant.

Fråga 2: Vad kännetecknar funktionell programmering?

Svar: Det bygger på att använda, kombinera och modifiera funktioner snarare än att styra programflödet med loopar och variabler som ändras.

Fråga 3: Varför behöver rekursion ett basfall?

Svar: Ett basfall behövs för att stoppa rekursionen och förhindra oändliga anrop.

```
def pascal(n):
'''
Ex:
1. pascal(3) exekveras:
```

```

    n = 3 (inte 1, gar vidare)
    Anropar pascal(2) for att fa 'prev_row'
2. pascal(2) exekveras:
    n = 2 (inte 1, gar vidare)
    Anropar pascal(1) for att fa 'prev_row'
3. pascal(1) exekveras:
    n = 1 (basfallet uppnatt!)
    RETURVARDE FRAN pascal(1): [1]

[RETURFASEN - Vagen upp och berakning av varden]
4. Tillbaka i pascal(2):
    Mottagit 'prev_row' = [1]
    Initierar 'new_elements' = []
    Beraknar loopen: range(len([1]) - 1) -> range(0) -> Loopen kors
        0 ganger
    Satter ihop raden: [1] + [] + [1]
    RETURVARDE FRAN pascal(2): [1, 1]
5. Tillbaka i pascal(3):
    Mottagit 'prev_row' = [1, 1]
    Initierar 'new_elements' = []
    Beraknar loopen: range(len([1, 1]) - 1) -> range(1) -> Loopen
        kors for i = 0
        * i = 0: prev_row[0] + prev_row[1] -> 1 + 1 = 2
        * new_elements.append(2) -> 'new_elements' ar nu [2]
    Satter ihop raden: [1] + [2] + [1]
    RETURVARDE FRAN pascal(3): [1, 2, 1]
,,,

    if n == 1:
        return [1]

    prev_row = pascal(n - 1)
    new_elements = []

    for i in range(len(prev_row) - 1):
        new_elements.append(prev_row[i] + prev_row[i+1])

    return [1] + new_elements + [1]

```

9 Kapitel 9: OOP och klasser

Objektorientering

är ett programmeringsparadigm eller programmeringsstil där man samlar data och dataskrukturer med de funktioner som kan tillämpas på dem.

Klass, objekt, instans

En klass är en mall för att skapa objekt. Ett objekt är en instans av en klass (dvs. ett konkret exempel på klassen). Skillnaden mellan objekt och instans är att objekt är en mer generell term, medan instans syftar på ett specifikt objekt som skapats från en klass.

Attribut

En variabel som är knuten till ett specifikt objekt och beskriver dess nuvarande tillstånd. De är som objektets variabler. Det finns instansattribut (knutna till ett specifikt objekt) och klassattribut (delade av alla objekt i klassen).

Metod

En funktion som är knuten till ett specifikt objekt och kan manipulera dess attribut eller

utföra operationer relaterade till objektet. De är som funktioner till objektet.

Konstruktor

En speciell metod som används för att skapa och initiera objekt av en klass. I Python är konstruktorn metoden `__init__()`. Så när man anropar klassen igen för att skapa ett nytt objekt, körs konstruktorn automatiskt för att sätta upp objektets initiala tillstånd.

Metod

En funktion som är knuten till ett specifikt objekt.

Punktnotation

Syntaxen `objekt.metod()` för att anropa metoder.

Self

En referens till det aktuella klassens objektet, används för att komma åt objektets attribut och metoder. Kan kallas annat men `self` är konvention.

Privata och publika attribut/metoder

Publika attribut/metoder kan nås utanför klassen, medan privata är avsedda att endast användas inom klassen. I Python markeras privata med ett understreck (`_`) i början av namnet.

Modulär programmering

Att dela upp program i mindre, fristående enheter (moduler) som kan återanvändas och underhållas separat vilket leder till mindre kod att ändra vid behov.

```
# Syntax for classes
class KlassNamn: # Bör namnges med stor bokstav i början, va ett
    substantiv tex Bilar

    competition = "Formula 1" # Klassattribut, delad av alla objekt
    i klassen

    def __init__(self, arg1, arg2): # Instansattribut, unik för
        varje objekt genom att data läggs till
        self.attribut1 = arg1 # Här skapas attribut/variabler tex å
            lder, vikt
        self.attribut2 = arg2

    def metod(self, attribut1, attribut2): # funktion/metod som ber
        äknar bilens värde
        värde = self.attribut1 * 1000 + self.attribut2 * 500 # Obs
            även enkla funktioner som len() behövs läggas till så de
            funkar på din klass
        return värde

    # Lägg till enkla funktioner vi vill ha
    def __str__(self): # __str__() är en speciell metod som
        definierar hur objektet ska representeras som en sträng
        return f"Objekt med attribut1: {self.attribut1}, attribut2:
            {self.attribut2}, competition: {KlassNamn.competition}"

# Syntax för att skapa ett objekt (instans) av klassen
objekt = KlassNamn(ålder1, vikt2) # objekt kan nu manipulera hela
    datan från Bilar med hjälp av enkel punktnotation
objekt.attribut3 = värde3 # Här kan man lägga till fler attribut,
    obs kan skrivas över
```



```

objekt.metod() # Här kan man anropa metoder som redan finns i
               klassen
objekt2 = KlassNamn(ålder2, vikt3) # Här skapas ett nytt objekt med
               annan data som en kopia
print(objekt2.attribut2) # För att använda funktioner som redan
               finns på objektet, kalla på dess info/attribut

```

```

# Privata och publika attribut/metoder
class Bil:
    def __init__(self, ålder, vikt):
        self._ålder = ålder # Privata attribut (bör inte ändras
                             utanför klassen)
        self._vikt = vikt   # Privata attribut (bör inte ändras
                             utanför klassen)

    def beräkna_värde(self):
        return self._ålder * 1000 + self._vikt * 500 # Publik
               metod som använder privata attribut

min_bil = Bil(5, 1500) # man kan manipulera med beräkna_värde() men
                       inte ändra _ålder eller _vikt direkt om man inte skriver...
min_bil._ålder = 10   # ...så här, men det är inte rekommenderat
                       eftersom det bryter mot principen om inkapsling.

```

```

# Sets
a = {1, 2, 3, 4, 5} # Vilket är samma som {1, 1, 2, 3, 4, 5, 5}
b = {4, 5, 6, 7, 8}
print(a | b) # Union
print(a & b) # Intersection
print(a - b) # Difference
print(a ^ b) # Symmetric difference = (a - b) | (b - a)

```

Fråga 1: Varför använda objektorientering?

Svar: Knyta samman mycket data som relaterar till varandra (hör till samma objekt) vilket ger bra struktur med relaterad funktionalitet på samma ställe. Också att kunna definiera egna datatyper som hjälper abstraktion genom att kunna gömma detaljer.

10 Kapitel 10: OOP arv

Arv

Mekanism för att återanvända och utöka kod genom subklasser.

Basklass

Den ursprungliga klassen som funktionalitet ärvs ifrån.

Subklass

Den klass som ärver från en basklass.

Fråga 1: Varför använder vi arv?

Svar: För att återanvända kod på ett strukturerat sätt och för att kunna bygga vidare på existerande klasser.

Fråga 2: Hur definieras en subklass i Python?

Svar: Man anger basklassen inom parentes i subklassens definition, t.ex. `class Sub(Bas):`.

11 Kapitel 11: OOP arv mer

Multipelt arv

När en klass ärver från två eller fler superklasser.

Diamantproblemet

En tvetydighet i arvshierarkin som kan uppstå vid multipelt arv.

Fråga 1: Vad är diamantproblemet?

Svar: Det är en konflikt i arvshierarkin där en subklass ärver från två klasser som delar samma basklass, vilket skapar osäkerhet om vilken metod som ska anropas.

Fråga 2: Varför är multipelt arv kontroversiellt?

Svar: Eftersom det kan göra arvshierarkin oöverskådlig och skapa konflikter som inte finns i enkla arvsstrukturer.

12 Kapitel 12: Defensiv programmering

Defensiv programmering

Ansats för att identifiera och hantera fel tidigt.

Assert

Kontroll av logiska antaganden i koden.

unittest

Modul för att automatisera och systematisera kodtestning.

Fråga 1: Varför använda assert?

Svar: För att verifiera att programmets tillstånd stämmer och fånga logiska fel tidigt.

Fråga 2: Vad gör unittest-modulen?

Svar: Den tillhandahåller ett ramverk för att organisera och köra automatiserade tester systematiskt.